Epreuve pratique - Sujet n°20

Exercice 1

Cet exercice ne présente aucune difficulté particulière. On utilise ici l'annotation de type vu dans le chapitre *Mise au point de programmes*, ce n'était **pas explicitement exigé** par le sujet. Il faut voir cette utilisation comme un prétexte de révision.

Il est raisonnable d'exiger comme préconditions que les tableaux passés en paramètres ne soient pas vides et qu'ils soient de même taille.

```python
from typing import List, Tuple


def mini(t_moy: List[float], annees: List[int]) -> Tuple[float, int]:
    """ Renvoie un tuple composé de la plus basse temperature et de l'année
correspondante"""

    assert len(t_moy) != 0 and len(annees) != 0, "Erreur liste(s) vide(s)"
    assert len(t_moy) == len(annees), "Erreur entree des données"
    idxmini = 0
    for i in range(1, len(t_moy)):
        if t_moy[i] < t_moy[idxmini]:
            idxmini = i
    return t_moy[idxmini], annees[idxmini]
```

```python
t_moy = [14.9, 13.3, 13.1, 12.5, 13.0, 13.6, 13.7]
annees = [2013, 2014, 2015, 2016, 2017, 2018, 2019]

mini(t_moy, annees)
```

```
(12.5, 2016)
```

⚠ Remarque n°1 - Hors programme

Python possède une fonction native `enumerate` -**non exigée à l'examen**- qui permet de réaliser la boucle `for` tout en ayant accès à l'élément **et** l'indice de l'élément.

```python
def mini_v2(t_moy: List[float], annees: List[int]) -> Tuple[float, int]:
    """ Renvoie un tuple composé de la plus basse temperature et de l'année
correspondante"""

    assert len(t_moy) != 0 and len(annees) != 0, "Erreur liste(s) vide(s)"
    assert len(t_moy) == len(annees), "Erreur entree des données"
    idxmini = 0
    for i, valeur in enumerate(t_moy):
        if valeur < t_moy[idxmini]:
            idxmini = i
    return t_moy[idxmini], annees[idxmini]
```

⚠ Remarque n°2 - Hors programme

On peut utiliser aussi la fonction `zip` qui permet d'itérer sur les deux listes simultanément. Cela va créer un itérateur qui produira un tuple des valeurs de `t_moy` et `annees`. La concision du programme obtenue est remarquable. L'instruction `lambda` permet de définir une fonction *anonyme*.

```python
mini_v3 = lambda t_moy, annees: min(zip(t_moy, annees))
```

⚠ Remarque n°3 - Hors programme

On peut comparer les performances des 3 solutions

```python
%timeit mini(t_moy, annees)
```

```
857 ns ± 2.05 ns per loop (mean ± std. dev. of 7 runs, 1000000 loops each)
```

```python
%timeit mini_v2(t_moy, annees)
```

```
803 ns ± 5.79 ns per loop (mean ± std. dev. of 7 runs, 1000000 loops each)
```

```python
%timeit mini_v3(t_moy, annees)
```

```
660 ns ± 3.26 ns per loop (mean ± std. dev. of 7 runs, 1000000 loops each)
```

Exercice 2

```python
def inverse_chaine(chaine):
    result = ""
    for caractere in chaine:
        result = caractere + result
    return result
```

```python
def est_palindrome(chaine):
    inverse = inverse_chaine(chaine)
    return inverse == chaine
```

```python
def est_nbre_palindrome(nbre):
    chaine = str(nbre)
    return est_palindrome(chaine)
```

```python
inverse_chaine('bac')
```

```
'cab'
```

```python
est_palindrome('NSI')
```

```
False
```

```python
est_palindrome('ISN-NSI')
```

```
True
```

```python
est_nbre_palindrome(214312)
```

```
False
```

```python
est_nbre_palindrome(213312)
```

```
True
```

Remarque

La construction `return inverse == chaine` évite les lourdeurs comme:

```
if inverse == chaine:
    return True
else:
    return False
```